TP3

Intervalles de confiance

Estimation d'une espérance et d'une proportion

Étudiants : CIPRIANI – VILLENEUVE
Date : 24 septembre 2026

Table des matières

1 Dissection d'une gaussienne **2**
1.1 Intervalle de confiance sur un petit échantillon 2
1.2 Intervalle de confiance sur un grand échantillon 2

2 Sondages **3**
2.1 Intervalles de confiance des intentions de vote 3
2.2 Taille d'échantillon pour le candidat Duval 4

3 Dommage de casques **5**
3.1 Intervalle de confiance de la proportion de casques endommagés 5
3.2 Taille d'échantillon à partir de l'estimation ponctuelle 5
3.3 Taille d'échantillon sans connaissance de p 5

4 ANNEXES **8**
4.1 Programme complet 8

1 Dissection d'une gaussienne

1.1 Intervalle de confiance sur un petit échantillon

Données : 0,82 ; 0,87 ; 0,77 ; 0,96 ; 0,75 ; 0,83 ; 0,87 ; 0,81, soit $n = 8$.

Hypothèses retenues.

- Les observations $X_1, \ldots, X_n$ sont indépendantes et identiquement distribuées selon une loi $\mathcal{N}(\mu,\sigma^2)$.
- L'espérance μ et l'écart-type σ sont inconnus : σ est estimé par l'écart-type corrigé S_{n-1}.
- Le niveau de confiance est fixé à $1 - \alpha = 95\,\%$.

On en déduit l'intervalle de confiance :

$$IC_{\mu,\alpha} = \left[\bar{x} - t_{\alpha/2,n-1}\frac{s_{n-1}}{\sqrt{n}} \; ; \; \bar{x} + t_{\alpha/2,n-1}\frac{s_{n-1}}{\sqrt{n}}\right].$$

L'échantillon étant petit ($n < 30$), on ne peut pas remplacer la loi de Student par la loi normale. Le calcul est réalisé avec `np.std(..., ddof=1)` pour obtenir s_{n-1} et `st.t.ppf` pour le quantile :

```
def interv_conf_inf_30(data, alpha):
    n = len(data)
    x_mean = np.mean(data)
    Ecart_type = np.std(data, ddof=1)
    fnct_repart_inv = st.t.ppf(1 - alpha/2, df=n-1)

    Int_CONF = np.array([x_mean - fnct_repart_inv*(Ecart_type/np.sqrt(n)),
                         x_mean + fnct_repart_inv*(Ecart_type/np.sqrt(n))])
    return Int_CONF

data = np.array([0.82, 0.87, 0.77, 0.96, 0.75, 0.83, 0.87, 0.81])
print(f"Intervalle de confiance : {interv_conf_inf_30(data, 0.05)}")
```

```
Intervalle de confiance : [0.77990592 0.89009408]
```

Cette fonction renvoie donc par vecteur numpy l'intervalle de confiance suivante :

$$IC_{95\%} \simeq [\,0{,}7799 \; ; \; 0{,}8901\,].$$

Si l'on répétait l'expérience un grand nombre de fois, 95 % des intervalles de confiance ainsi construits contiendraient la vraie moyenne μ de la population, c'est-à-dire la moyenne réelle de la grandeur étudiée.

1.2 Intervalle de confiance sur un grand échantillon

On dispose maintenant de $n = 40$ observations.

Hypothèses retenues.

- Les observations sont indépendantes et identiquement distribuées, d'espérance μ et de variance σ^2 finie.
- σ est inconnu et estimé à partir de l'échantillon.

— L'échantillon est suffisamment grand ($n \geq 30$) pour utiliser l'approximation normale.
— Le niveau de confiance est fixé à 95 %.

On utilise alors :

$$IC_{\mu,\alpha} = \left[\bar{x} - z_{\alpha/2}\frac{s_n}{\sqrt{n}} \; ; \; \bar{x} + z_{\alpha/2}\frac{s_n}{\sqrt{n}}\right].$$

```
def interv_conf_grand_nombres(data, alpha):
    n = len(data)
    x_mean = np.mean(data)
    Ecart_type = np.std(data)
    Z_alpha_2 = st.norm.ppf(1 - alpha/2)

    Int_CONF = np.array([x_mean - Z_alpha_2*(Ecart_type/np.sqrt(n)),
                         x_mean + Z_alpha_2*(Ecart_type/np.sqrt(n)) ])
    return Int_CONF
```

```
Intervalle de confiance : [0.75517633 0.81582367]
```

$$IC_{95\%} \simeq [\,0{,}7552 \; ; \; 0{,}8158\,].$$

Intérêt de la loi gaussienne : Pour un grand échantillon, le théorème central limite assure que $\bar{X}$ suit approximativement une loi normale, *quelle que soit la loi des observations*. L'hypothèse gaussienne sur les données n'est donc plus indispensable : la loi normale centrée réduite sert de loi limite à la variable pivot. De plus, la loi de Student converge vers la loi normale quand n augmente : $t_{0.975,39} = 2{,}0227$ contre $z_{0.975} = 1{,}9600$. On peut donc utiliser directement le quantile gaussien, qui ne dépend pas de n.

On constate aussi que l'intervalle est environ deux fois plus étroit qu'à la question précédente (0,0303 contre 0,0551 de demi-largeur).

2 Sondages

2.1 Intervalles de confiance des intentions de vote

Sur $n = 1000$ personnes interrogées, 500 déclarent voter Dupond, 250 Durand et 50 Duroc. Les 200 personnes restantes n'étant pas attribuées à ces trois candidats, la proportion est calculée sur l'effectif total $n = 1000$.

L'échantillon étant grand ($n \geq 30$), l'intervalle de confiance à 95 % et 99 % de la proportion p est alors donné par :

$$\hat{p} - z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \; \leq p \leq \; \hat{p} + z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Cette approximation est valable car $n\hat{p} \geq 5$ et $n(1-\hat{p}) \geq 5$ pour les trois candidats (le plus petit effectif est 50 pour Duroc).

```
def interv_conf_prop_grand_echantillon(data, component, percent, n):
    alpha = 1-percent
    prop_data = np.sum(data == component) / n
    Z_alpha_2 = st.norm.ppf(1 - alpha/2)
    I_C = np.array([prop_data - Z_alpha_2 * np.sqrt((prop_data * (1 -
        prop_data))/n),
```

```
                    prop_data + Z_alpha_2 * np.sqrt((prop_data * (1 -
                        prop_data))/n)])
    return I_C

n = 1000
data = np.array(
    ["Dupond"] * int(500) +
    ["Durand"] * int(250) +
    ["Duroc"] * int(50)
)

for confiance in [0.95, 0.99]:
    print(f"\nIntervalle de confiance à {confiance * 100:.0f} %")
    for candidat in np.unique(data):
        IC = interv_conf_prop_grand_echantillon(data, candidat,
            confiance, n)
        print(f"{candidat:<8} : [{IC[0]:.4f} ; {IC[1]:.4f}]")
```

```
Intervalle de confiance à 95 %
Dupond   : [0.4690 ; 0.5310]
Durand   : [0.2232 ; 0.2768]
Duroc    : [0.0365 ; 0.0635]

Intervalle de confiance à 99 %
Dupond   : [0.4593 ; 0.5407]
Durand   : [0.2147 ; 0.2853]
Duroc    : [0.0322 ; 0.0678]
```

Les quantiles utilisés sont $z_{0,975} = 1{,}9600$ et $z_{0,995} = 2{,}5758$.

Candidat	$\hat{p}$	$IC_{95\%}$	$IC_{99\%}$
Dupond	50 %	[46,90 % ; 53,10 %]	[45,93 % ; 54,07 %]
Durand	25 %	[22,32 % ; 27,68 %]	[21,47 % ; 28,53 %]
Duroc	5 %	[3,65 % ; 6,35 %]	[3,22 % ; 6,78 %]

Les intervalles à 99 % sont plus larges que ceux à 95 % : augmenter la confiance se paie par une perte de précision.

2.2 Taille d'échantillon pour le candidat Duval

On souhaite une précision (demi-largeur) $E = 1\,\%$ pour un intervalle de confiance à 95 % avec $\hat{p} = 0{,}17$. On impose :

$$n \geq \left(\frac{z_{\alpha/2}}{E}\right)^2 \hat{p}(1-\hat{p}).$$

```
n_calc = np.ceil(((st.norm.ppf(1 - 0.05/2)/0.01)**2)*0.17*(1-0.17))
print(f"Quantité n arrondi nécessaire : {n_calc}")
```

```
Quantité n arrondi nécessaire : 5421.0
```

$$n \geq \left(\frac{1{,}96}{0{,}01}\right)^2 \times 0{,}17 \times 0{,}83 \simeq 5420{,}30 \quad \Longrightarrow \quad n = 5421.$$

L'arrondi se fait à l'entier supérieur (`np.ceil`) : avec 5420 personnes, la demi-largeur dépasserait légèrement 1 %. Il faut donc interroger au moins **5421 personnes**.

3 Dommage de casques

3.1 Intervalle de confiance de la proportion de casques endommagés

Sur $n = 50$ casques testés, 18 sont endommagés, soit $\hat{p} = 18/50 = 0{,}36$. On a $n\hat{p} = 18 \geq 5$ et $n(1-\hat{p}) = 32 \geq 5$: l'approximation normale de la section précédente s'applique et on réutilise la même fonction.

```
n = 50
abime = 18
data = np.array(
    ["Casque_abime"] * abime +
    ["casque_normal"] * (n - abime)
)
casques = np.unique(data)
prop_data = np.sum(data == casques[0]) / n
print(prop_data)
print(f"{casques[0]:<8} : {interv_conf_prop_grand_echantillon(data,
    casques[0], 0.95, n)}")
```

```
0.36
Casque_abime : [0.22695323 0.49304677]
```

$$0{,}36 \pm 1{,}96\sqrt{\frac{0{,}36 \times 0{,}64}{50}} = 0{,}36 \pm 0{,}1330 \quad \Longrightarrow \quad IC_{95\%}(p) \simeq [\,0{,}2270\ ;\ 0{,}4930\,].$$

Avec seulement 50 casques, l'intervalle est large (environ 27 points) : la proportion réelle de casques endommagés se situe entre environ 23 % et 49 %.

3.2 Taille d'échantillon à partir de l'estimation ponctuelle

On cherche le nombre de casque à tester afin d'obtenir une erreur E inférieur à 0,02 pour l'IC à 95 %, en utilisant l'estimateur ponctuel $\hat{p} = 0{,}36$ obtenu sur les 50 casques :

$$n \geq \left(\frac{z_{\alpha/2}}{E}\right)^2 \hat{p}(1-\hat{p}) = \left(\frac{1{,}96}{0{,}02}\right)^2 \times 0{,}36 \times 0{,}64 \simeq 2212{,}68.$$

```
n_calc = np.ceil(((st.norm.ppf(1 - 0.05/2)/0.02)**2)*prop_data*(1-
    prop_data))
print(f"Quantité n arrondi nécessaire : {n_calc}")
```

```
Quantité n arrondi nécessaire : 2213.0
```

Il faut donc tester au moins **2213 casques**.

3.3 Taille d'échantillon sans connaissance de p

Sans aucune information sur p ni sur $\hat{p}$, on se place dans le cas le plus défavorable, c'est-à-dire celui qui maximise la taille requise :

$$n = f(p) = \left(\frac{z_{\alpha/2}}{E}\right)^2 p(1-p).$$

Le facteur $\left(z_{\alpha/2}/E\right)^2$ étant constant, il suffit de maximiser $g(p) = p(1-p)$ sur $[0\ ;\ 1]$:

$$g'(p) = 1 - 2p = 0 \iff p = \frac{1}{2}, \qquad g''(p) = -2 < 0.$$

Le maximum est atteint en $p = 1/2$, où $g(1/2) = 1/4$. On obtient :

$$n \geq \left(\frac{1{,}96}{0{,}02}\right)^2 \times \frac{1}{4} \simeq 2400{,}91 \quad \Longrightarrow \quad n = 2401.$$

```python
alpha = 0.05
erreur = 0.02

p = 0.5  # valeur qui maximise p * (1 - p)
z_alpha_2 = st.norm.ppf(1 - alpha / 2)

n = np.ceil(
    (z_alpha_2 / erreur)**2 * p * (1 - p)
)

print("Valeur de p maximisant n :", p)
print("Taille minimale de l'échantillon :", int(n))

p_val = np.linspace(0, 1, 500)
n_val = (z_alpha_2 / erreur)**2 * p_val * (1 - p_val)
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(p_val, n_val)
ax.axvline(0.5, color="red", linestyle="--", label="p = 0.5")
ax.set_xlabel("Proportion p")
ax.set_ylabel("Taille d'échantillon n")
```

```
Valeur de p maximisant n : 0.5
Taille minimale de l'échantillon : 2401
```

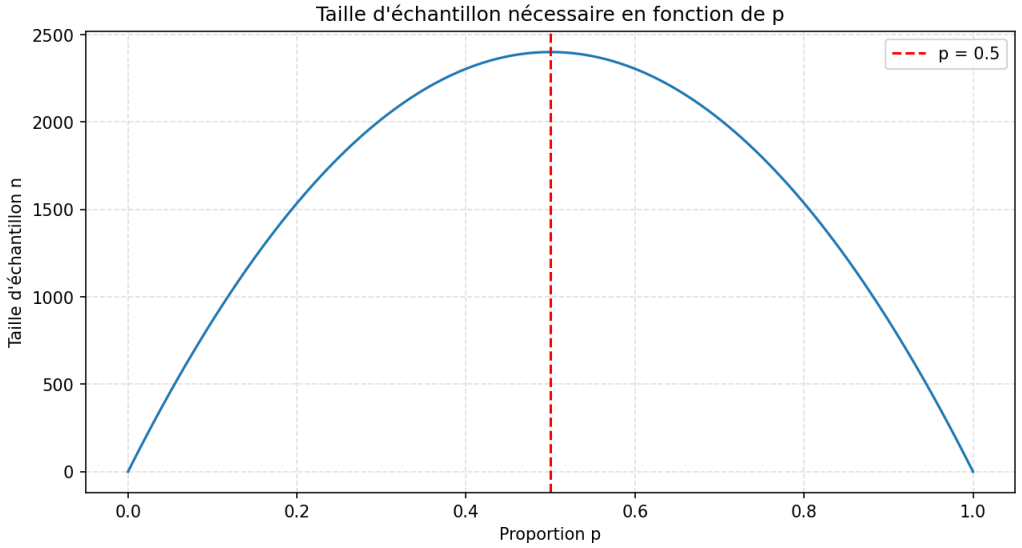


FIGURE 1 – Taille d'échantillon nécessaire $n = f(p)$ pour une erreur de 0,02 à 95 %.

La courbe est une parabole symétrique autour de $p = 0{,}5$. Il faut donc tester au moins **2401 casques** pour garantir une erreur inférieure à 0,02 quelle que soit la vraie valeur de

p. Cette valeur est plus grande que les 2213 casques de la question précédente : l'absence d'information sur p impose de se placer dans le pire cas.

4 ANNEXES

4.1 Programme complet

```
import numpy as np
import scipy.stats as st
import matplotlib.pyplot as plt

def interv_conf_inf_30(data, alpha):
    #Intervalle de confiance
    n = len(data)
    x_mean = np.mean(data)
    Ecart_type = np.std(data, ddof=1)
    fnct_repart_inv = st.t.ppf(1 - alpha/2, df=n-1)

    Int_CONF = np.array([x_mean - fnct_repart_inv*(Ecart_type/np.sqrt(n)),
                         x_mean + fnct_repart_inv*(Ecart_type/np.sqrt(n))])
    return Int_CONF


def interv_conf_grand_nombres(data, alpha):
    #Intervalle de confiance
    n = len(data)
    x_mean = np.mean(data)
    Ecart_type = np.std(data)
    Z_alpha_2 = st.norm.ppf(1 - alpha/2)

    Int_CONF = np.array([x_mean - Z_alpha_2*(Ecart_type/np.sqrt(n)),
                         x_mean + Z_alpha_2*(Ecart_type/np.sqrt(n)) ])
    return Int_CONF

def interv_conf_prop_grand_echantillon(data, component, percent, n):
    alpha = 1-percent
    prop_data = np.sum(data == component) / n
    Z_alpha_2 = st.norm.ppf(1 - alpha/2)
    I_C = np.array([prop_data - Z_alpha_2 * np.sqrt((prop_data * (1 - prop_data))/n),
                    prop_data + Z_alpha_2 * np.sqrt((prop_data * (1 - prop_data))/n)])
    return I_C

if __name__ == "__main__":

    print("////////////////////////////////////////////")
    print("PARTIE 2")
    print("Question 2.1")
    data = np.array([0.82, 0.87, 0.77, 0.96, 0.75, 0.83, 0.87, 0.81])
    print(f"Intervalle de confiance : {interv_conf_inf_30(data, 0.05)}")

    print("Question 2.2")
    data = np.array([
    0.84, 0.87, 0.89, 0.73, 0.84, 0.81, 0.88, 0.85,
    0.89, 0.79, 0.79, 0.90, 0.59, 0.75, 0.67, 0.76,
    0.86, 0.88, 0.70, 0.75, 0.81, 0.77, 0.83, 0.84,
    0.71, 0.78, 0.59, 0.91, 0.74, 0.68, 0.77, 0.66,
    0.80, 0.74, 1.02, 0.91, 0.55, 0.84, 0.66, 0.77
```

```
    ])
    print(f"Intervalle de confiance : {interv_conf_grand_nombres(data,
        0.05)}")

    print("\n")
    print("////////////////////////////////////////////")
    print("PARTIE 3")
    print("Question 3.1")

    n = 1000
    data = np.array(
        ["Dupond"] * int(500) +
        ["Durand"] * int(250) +
        ["Duroc"] * int(50)
    )

    for confiance in [0.95, 0.99]:
        print(f"\nIntervalle de confiance à {confiance * 100:.0f} %")
        for candidat in np.unique(data):

            IC = interv_conf_prop_grand_echantillon(
                data, candidat, confiance, n
            )
            print(f"{candidat:<8} : [{IC[0]:.4f} ; {IC[1]:.4f}]")

    print("Question 3.2")
    #Formule du cours slide 144
    n_calc = np.ceil(((st.norm.ppf(1 - 0.05/2)/0.01)**2)*0.17*(1-0.17))
    print(f"Quantité n arrondi nécessaire : {n_calc}")
    print("\n")
    print("////////////////////////////////////////////")
    print("PARTIE 4")
    print("Question 4.1")
    n = 50
    abime = 18
    data = np.array(
        ["Casque_abime"] * abime +
        ["casque_normal"] * (n - abime)
    )
    casques = np.unique(data)
    prop_data = np.sum(data == casques[0]) / n
    print(prop_data)
    print(f"{casques[0]:<8} : {interv_conf_prop_grand_echantillon(data,
        casques[0], 0.95, n)}")

    print("Question 4.2")
    n_calc = np.ceil(((st.norm.ppf(1 - 0.05/2)/0.02)**2)*prop_data*(1-
        prop_data))
    print(f"Quantité n arrondi nécessaire : {n_calc}")

    print("Question 4.3")

    alpha = 0.05
    erreur = 0.02

    p = 0.5  # valeur qui maximise p * (1 - p)
    z_alpha_2 = st.norm.ppf(1 - alpha / 2)

```

```
    n = np.ceil(
        (z_alpha_2 / erreur)**2 * p * (1 - p)
    )

    print("Valeur de p maximisant n :", p)
    print("Taille minimale de l'échantillon :", int(n))

    p_val = np.linspace(0, 1, 500)
    n_val = (z_alpha_2 / erreur)**2 * p_val * (1 - p_val)
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(p_val, n_val)
    ax.axvline(0.5, color="red", linestyle="--", label="p = 0.5")
    ax.set_xlabel("Proportion p")
    ax.set_ylabel("Taille d'échantillon n")
    ax.set_title("Taille d'échantillon nécessaire en fonction de p")
    ax.grid(linestyle="--", alpha=0.4)
    ax.legend()
    fig.savefig("fig_n_p.png", dpi=150, bbox_inches="tight")
    plt.show()
```